Gnkc Hackathon 2026

Research Work:-

Tumhari strategy workable hai—but win karne ke liye AI ko "one-shot app generator" nahi, fast implementation assistant ki tarah use karna hoga. Tumhare case mein best plan hoga: ek hi person product ownership aur integration kare, team ko presentation/testing/research roles do, aur problem reveal ke baad scope ko ruthlessly small rakho.

Hackathon mein 27–30 August ka preparation window tumhari biggest advantage hai: pre-hackathon mein 60–70% app complete karo, phir 31 August ko sirf polish, testing, deployment, aur pitch.

Tumhari team strategy

Tum 4 log ho, female member confirm hai, aur practical reality ye hai ki tum primary builder hoge. Lekin judges ke saamne "main hi sab kuch kar raha tha" bolna avoid karo. Team ko credible roles do—har person actual useful output de.

Team member	Practical role	Deliverable
Tum	Product lead + full-stack integration	Architecture, Supabase, frontend, deployment, final integration
Member 2	Problem research + documentation	Problem analysis, target users, competitor notes, feature list
Member 3	QA + demo-data manager	Test cases, bug list, realistic sample data, screenshots/video backup
Female member	UX reviewer + presentation co-lead	UI feedback, user journey, presentation script, judge interactions

Yeh division fake nahi hai. Agar har member apna deliverable genuinely complete kare, toh team project hi rahega—even if code mostly tum build kar rahe ho.

Important: Tumhara role

Tumhara strongest advantage tumhari existing workflow hai:

> **Brainstorming → documentation → database → backend → frontend → security review → deployment.**

Is process ko abandon mat karo; bas hackathon ke liye compressed and more disciplined version use karo. 4 din mein documentation ka goal "perfect documentation" nahi, AI ko correct context dena aur tumhe scope se bhatakne se bachana hai.

Recommended stack

Tumhare experience aur AI-assisted workflow ke basis par:

Layer	Recommended choice	Why
Frontend	React + Vite + Tailwind CSS	Fast UI building, components reusable, AI tools support well
Backend + database + auth	Supabase	PostgreSQL database, authentication, storage, APIs aur dashboard ek jagah
Hosting	Vercel	React frontend deployment comparatively straightforward
Version control	Existing GitHub repository	Backup, rollback aur project proof
AI builder	Antigravity	UI build, debugging aur feature implementation
Research/product planning	Perplexity + Gemini	Problem validation, PRD, schema, user journeys

Agar React/Vite mein setup ya build errors ka risk lage, toh Next.js + Supabase bhi good option hai, but first hackathon ke liye React + Vite usually simpler rahega. Backend ko separate Node/Express mein tabhi banao jab problem statement ko truly complex server-side logic ya external APIs chahiye. Most college-hackathon MVPs ke liye Supabase directly enough hoga.

Stack rule

> **Supabase + React + Vercel se bahar tabhi jao jab problem statement tumhe force kare.**

Ek additional backend, multiple databases, complex microservices, payment gateway, live AI model pipeline—ye sab time waste aur failure risk badhate hain.

27 August: Problem choose karne ka system

10 AM ko statements reveal hone ke baad excitement mein pehla “cool AI idea” choose mat karna. Pehle dono ko score do.

Criteria	Weight	What a high score looks like
4 din mein working MVP	25	One primary user flow, limited screens, manageable data
Live demo impact	20	Judge directly input de aur visible useful output dekhe
Supabase fit	15	Users, records, forms, dashboard, CRUD, auth/data workflow
UI/UX potential	15	App visually polished and easy to navigate
Originality	10	Specific differentiated feature—not generic chatbot
Scalability story	10	Can explain expansion to institutions/users/cities
External dependency risk	5	Works even without costly/unreliable APIs

Har problem ko 1–5 score do. Final calculation:

Problem score=∑(score×weight)\text{Problem score} = \sum(\text{score} \times \text{weight})Problem score=∑(score×weight)

Kaunsa type ka problem generally choose karna chahiye

Agar choices mein koi workflow-management / dashboard / matching / reporting / verification / tracking category ka problem ho, usse preference do.

Examples:

- **Student opportunity, scholarship ya internship tracking.**
- **Local civic issue reporting and resolution tracking.**
- **College event/resource/attendance management.**
- **Volunteer-donor-beneficiary matching.**
- **Healthcare appointment/follow-up tracker.**
- **Waste collection/reporting workflow.**
- **Accessibility-oriented reporting or assistance platform.**

Aise problems ke pros:

- **Supabase ke saath naturally fit hote hain.**

- **Demo data bana sakte ho.**
- **Strong dashboards and UI bante hain.**
- **Auth, roles, forms, status tracking, analytics jaise visible features demonstrate kar sakte ho.**
- **“Future scale” clearly explain hota hai.**

High-risk problem patterns avoid karo

Problem Statement 2 choose karo: “Escape the PDF Prison.”

Tumhari team ke current skill level, AI-assisted workflow, Supabase preference, limited time, aur presentation impact ko dekhte hue Problem 2 zyada controllable aur demo-friendly hai. Problem 1 technically impressive ho sakta hai, lekin usmein OCR, poor-quality image processing, extraction accuracy aur document comparison ka risk kaafi zyada hai.

Dono problems ka comparison

Criteria	Problem 1: Document Rescue	Problem 2: Resume to Portfolio
MVP 4 din mein complete karna	Medium–Low	High
AI tools ke saath implementation	Medium	High
Supabase fit	High	High
UI/UX presentation impact	High	Very high
External API/OCR dependency	High	Medium
Demo reliability	Medium–Low	High
Unique features add karna	High	High
Technical failure risk	High	Medium
Tumhare current workflow ke liye suitability	Medium	Very high

Problem 1: Pros and cons

Pros

Problem 1 ka goal difficult document images ko structured, reviewable information mein convert karna hai. Ismein multi-image upload, readable

output, intelligent extraction, correction, export aur original-vs-processed comparison expected hai.

Is problem ke advantages:

- **Real-world banking/lending use case hai.**
- **OCR, document quality score aur verification confidence jaise features impressive lag sakte hain.**
- **Multi-document processing se strong innovation story ban sakti hai.**
- **Judges ko visible before/after demo dikha sakte ho.**
- **Optional features jaise unclear-document detection aur human verification strong brownie points de sakte hain.**

Cons

Tumhari current team ke liye major risks:

- **OCR ke liye external service ya complex processing chahiye.**
- **Blurred, tilted, cropped documents par extraction unreliable ho sakta hai.**
- **Different layouts ke liye parser banana difficult hai.**
- **Personal identity documents handle karne padenge, jisse privacy/security questions aayenge.**
- **"Reliable structured information" prove karna difficult ho sakta hai.**
- **Agar live API fail hui, toh poora main demo weak ho jayega.**
- **Judges extraction accuracy, false data aur verification process par deep questions pooch sakte hain.**

Verdict: Problem 1 tab choose karo jab team mein kisi ko OCR/document AI ka genuine experience ho ya reliable OCR API immediately available ho.

Problem 2: Pros and cons

Pros

Problem 2 resume ko editable aur shareable professional portfolio mein transform karne ke baare mein hai. Ismein authentication, resume

upload, structured information, portfolio templates, editing, multiple sections, responsive design aur unique public link expected hai.

Is problem ke advantages:

- **Tumhara existing website/frontend experience directly apply hota hai.**
- **React + Supabase + Vercel stack ke saath naturally fit hota hai.**
- **Resume upload ke liye initially structured form ya controlled extraction use karke risk reduce kar sakte ho.**
- **Final output visually impressive hoga.**
- **Judges ko live public portfolio link dikha sakte ho.**
- **CRUD, authentication, edit/reorder, templates aur publishing clearly demonstrate kiye ja sakte hain.**
- **Demo predictable hai: user data → portfolio preview → edit → publish → public link.**
- **Personal portfolio use case tum khud easily understand aur explain kar sakte ho.**
- **Future scope strong hai: AI bio improvement, quality score, analytics, multiple themes, PDF export.**

Cons

- **Generic resume builder bahut common idea lag sakta hai.**
- **Resume-to-portfolio conversion sirf form-based hua toh innovation score kam ho sakta hai.**
- **Authentication, file parsing, editor, templates aur public links ek saath scope badha sakte hain.**
- **Multiple layout templates banane mein UI effort lagega.**
- **Resume extraction galat hone par user ko manual correction chahiye.**
- **Public portfolio privacy controls carefully design karne honge.**

Verdict: Problem 2 choose karo, lekin ise simple "resume website generator" mat banao. Iska stronger product angle banao.

Strong product concept

Suggested name

PortfolioForge
Tagline:

> **From one resume to a professional online presence in minutes.**

Alternative team-branded names:

- **StackFolio**
- **ResumeRise**
- **CareerCanvas**
- **ProfilePilot**
- **LaunchFolio**

Mera favourite: StackFolio by Team Stack Attack.

Core value proposition

> **Students and freshers often have skills and projects but lack the technical ability to create a professional online portfolio. StackFolio converts their resume into an editable, responsive and shareable portfolio that they can publish without coding.**

Ye angle judges ko simple aur relatable lagega.

Product ka winning MVP

Tumhe saare listed features ek saath nahi banane. MVP ko is order mein build karo.

Must-have features

1. **Demo login**
    - **Real authentication ya pre-created demo account.**
    - **Agar time kam ho toh landing page + demo mode use karo, but secure architecture explain karo.**
2. **Resume/profile input**
    - **Resume upload optional rakho.**
    - **Reliable demo ke liye “paste resume text” ya structured onboarding form zaroor rakho.**
    - **User name, headline, about, education, skills, projects, achievements, links.**

3. **Structured editor**
    - **Profile edit.**
    - **Projects add/edit/delete.**
    - **Skills update.**
    - **Education and achievements manage.**
    - **Sections reorder karne ke liye simple up/down buttons enough hain; drag-and-drop mandatory nahi.**
4. **Two polished portfolio templates**
    - **Template 1: Modern dark/blue developer layout.**
    - **Template 2: Clean professional light layout.**
    - **Teen ya chaar templates banane ke chakkar mein quality compromise mat karo.**
5. **Live preview**
    - **Editor ke saath portfolio preview.**
    - **Changes ka visible effect.**
6. **Publish feature**
    - **Unique slug, for example:** `yourdomain.com/p/rahul-sharma`
    - **Public portfolio page.**
    - **Publish/unpublish toggle.**
7. **Responsive design**
    - **Laptop aur mobile dono par clean layout.**
    - **Ye problem statement ka direct requirement hai.**

Brownie-point feature

Sabse practical differentiator:

Portfolio Readiness Score

App profile ko check karke feedback de:

- **Profile photo missing.**
- **About section too short.**
- **No project links.**
- **Skills listed but projects absent.**
- **No achievement or certification.**
- **Contact/social links incomplete.**

Example:

```text
Portfolio readiness: 72/100

Improve these areas:
1. Add one measurable project outcome.
2. Add GitHub or live demo link.
3. Write a longer professional introduction.
```

Ye feature:

- **Easy to demonstrate hai.**
- **AI ke bina rule-based ban sakta hai.**
- **AI add karoge toh optional enhancement ban jayega.**
- **Judges ko actionable value dikhegi.**
- **“Portfolio quality feedback” brownie point directly cover karta hai.Gnkc-P.S-2.pdf**

Features jo abhi mat banao

- **Payment.**
- **Complex drag-and-drop page builder.**
- **Full resume OCR/parser pipeline.**
- **Multiple admin roles.**
- **Portfolio visitor analytics, unless core app complete ho.**
- **PDF export, unless last day par easy library se add ho.**
- **Ten templates.**
- **AI chatbot.**
- **Full LinkedIn import.**
- **Real-time collaboration.**

Resume extraction ka practical solution

Problem statement resume upload aur structured extraction expect karta hai. Isliye upload feature completely skip mat karna. Lekin reliable MVP ke liye fallback rakho.

Recommended flow

```text
Upload PDF
   ↓
Extract text
   ↓
Show editable structured draft
   ↓
User corrects details
   ↓
Generate portfolio
```

Agar PDF parser ya AI extraction unreliable ho:

```text
Upload resume
   ↓
Use demo extraction / paste extracted text
   ↓
User confirms and edits fields
   ↓
Generate portfolio
```

Judges ke saamne honestly explain karna:

> **“Our design treats extraction as an assisted process. We do not blindly publish parsed content; the user reviews and edits the structured draft before publication.”**

Ye statement trust, safety aur usability tino demonstrate karega.

AI feature ka safe use

AI se entire resume parsing par depend mat karo. AI ko sirf optional enhancement ke liye use karo:

- **Professional summary generate karna.**
- **Project description improve karna.**
- **Portfolio completeness suggestions.**
- **Skills ko project evidence se compare karna.**

Core portfolio generation deterministic rakho, taaki demo reliable rahe.

Exact architecture

```text
React + Vite + Tailwind
        ↓
Supabase Auth
        ↓
Supabase PostgreSQL
        ↓
Supabase Storage for resume files
        ↓
Public portfolio route using username/slug
        ↓
Vercel deployment
```

Suggested database tables

`profiles`

```text
id
user_id
full_name
headline
bio
profile_image_url
location
email
github_url
linkedin_url
selected_template
is_published
public_slug
created_at
updated_at
```

experiences

```text
id
profile_id
company
role
start_date
end_date
description
display_order
created_at
```

education

```text
id
profile_id
institution
degree
field
start_year
end_year
description
display_order
```

projects

```text
id
profile_id
title
description
technologies
github_url
live_url
image_url
display_order
created_at
```

skills

```text
id
profile_id
name
category
level
display_order
```

achievements

```text
id
profile_id
title
issuer
date
description
credential_url
display_order
```

resume_uploads

```text
id
profile_id
file_name
file_url
extracted_text
processing_status
created_at
```

portfolio_feedback

```text
id
profile_id
score
suggestions
created_at
```

Important database decision

Initially tum nested JSON ke saath ek hi `profiles` table bana sakte ho, lekin judges ke saamne normalized tables zyada professional lagenge. Projects, education, skills aur experience separate tables rakho, kyunki:

- **Multiple entries easily support hongi.**
- **Add/edit/delete simple hoga.**
- **Reordering possible hogi.**
- **Schema scalable lagega.**
- **Supabase queries clear rahengi.**

Ye files banao:

```text
/docs
  PRD.md
  Architecture.md
  database-schema.md
  api-contract.md
  design.md
  phases.md
  test-plan.md
  demo-script.md
  security-checklist.md
  AGENTS.md
```

Supabase

- **Supabase project create.**
- **Tables create.**
- **Storage bucket create.**
- **Auth enable.**
- **RLS policies enable.**
- **Demo user/profile data add.**
- **Connection test.**

Remaining work

- **Landing page.**
- **Dashboard.**
- **Editor.**
- **Portfolio preview.**
- **Public profile route.**

- **Template switch.**
- **Readiness score.**

Important scope decision: upload vs editor

Problem statement ka full expectation resume upload hai, but tumhara winning MVP upload + editable correction + publish hona chahiye—not perfect automated parsing.

Priority:

1. **Portfolio generation.**
2. **Editing.**
3. **Public share link.**
4. **Responsive template.**
5. **Resume upload.**
6. **Extraction.**
7. **AI suggestions.**

Agar tum upload/extraction pe pehle 6 hours laga doge aur portfolio output incomplete reh gaya, score heavily hurt ho sakta hai. Functionality and completion ko 25 marks aur UX ko 25 marks diye gaye hain, isliye polished end-to-end flow zyada important hai.

Judges ke table visit ke liye pitch

English mein simple explanation ready rakho:

> **"Good morning, sir/ma'am. We are Team Stack Attack, and our product is StackFolio. Students often have valuable projects and achievements, but their information remains locked inside a static resume. StackFolio converts this information into an editable, responsive and shareable professional portfolio. The user can review their profile, choose a visual template, improve their content and publish a unique portfolio link. We are using React for the frontend and Supabase for authentication, database and storage."**

Phir immediately flow dikhao:

1. **Existing profile/resume open karo.**
2. **Edit a project.**

3. **Switch template.**
4. **Show readiness score.**
5. **Publish portfolio.**
6. **Open public share link in another tab.**

Agar judge pooche: “Resume upload kaha hai?”

Answer:

> **“The uploaded resume is treated as a draft source. We extract or organize the information into editable fields, and the user reviews it before publishing. This prevents incorrect information from going directly to the public portfolio.”**

Agar judge pooche: “Aapka unique feature kya hai?”

Answer:

> **“We are not only converting a resume into a webpage. We are giving users control over the entire transformation—editable structured data, multiple layouts, a publishable link and a portfolio readiness score that tells them exactly what to improve.”**

Agar judge pooche: “AI kaha use kiya?”

Answer:

> **“AI is used as an assistive layer for content suggestions and information organization, but the user remains in control. The final structured profile is always editable and reviewable before publication.”**

Final decision

Problem 2 lo. Product ka scope rakho:

> **Resume/profile input → editable structured data → polished portfolio → readiness feedback → unique public link**

Aur tumhari team ka differentiator rakho:

> **“Not just a resume parser—an editable, guided portfolio publishing system for students and freshers.”**

Ab tumhara immediate task hai: Problem 2 ke liye `StackFolio` ya preferred name lock karna, must-have features freeze karna, aur Supabase schema create karna. Main tumhare liye next step mein directly complete `PRD.md`, `database-schema.md`, `Architecture.md`, `api-contract.md`, `phases.md`, `design.md`, `AGENTS.md` aur Antigravity master prompt prepare kar sakta hoon.

Great decision. Problem Statement 2 is not merely a “resume website maker”; it is a career-identity platform that converts scattered professional information into a portfolio a student can confidently share.

The objective is to take a static resume, organize it into editable career data, generate a polished responsive portfolio, and publish it through a unique link. The statement specifically expects login, resume upload/extraction, editable sections, selectable layouts, responsiveness, and a shareable public portfolio URL.

Problem ko deeply samjho

Current user problem

Target users: college students, freshers, interns, freelancers, and early-career professionals.

A student often has:

- **Resume PDF stored on their phone or Drive.**
- **Projects scattered across GitHub, Google Drive, LinkedIn, and WhatsApp.**
- **Certificates in different folders.**
- **Skills listed in a resume but no proof or presentation.**
- **No personal portfolio because coding, hosting, design, and writing all feel difficult.**

A static PDF resume has limitations:

Resume PDF	Digital Portfolio
Fixed layout	Editable and continuously updatable
Usually one or two pages	Can show projects, links, visuals, and detailed work
Hard to personalize	Multiple design templates possible
Shared as a file	Shared as a professional public URL
Skills are only claims	Projects, demos, GitHub and achievements can support claims
Gets outdated quickly	User can update individual sections anytime

Product ka core insight

> **A resume contains career information, but it does not automatically create a strong professional presence.**

Tumhara product users ko coding seekhaye bina resume-based portfolio publish karne deta hai.

Product vision

Suggested product name

StackFolio

> ***Turn your resume into a portfolio that gets noticed.***

Other options:

- **CareerCanvas**
- **FolioFlow**
- **ProfilePilot**
- **LaunchFolio**
- **ResumeRise**
- **Portfolia**

Mere hisaab se StackFolio Team Stack Attack ke saath best branding banata hai.

One-line pitch

StackFolio transforms a static resume into an editable, responsive and shareable professional portfolio—without requiring the user to code or design.

Problem statement in formal form

Students and freshers often have valuable skills, projects and achievements, but these remain locked in static resumes that are difficult to showcase professionally. Building a personal portfolio requires design, technical and content-writing effort, creating a gap between a person's capability and their online professional presence.

Proposed solution

StackFolio allows users to upload or enter resume information, review and organize it into editable sections, select a professional template, receive actionable portfolio feedback, and publish a unique portfolio link.

User personas

Tumhe 3 personas identify karne chahiye. Presentation mein primary persona use karna.

Persona 1: Primary user — student/fresher

Name: Aarya Shah
Age: 20
Profile: Final-year BCA student
Goal: Internship/job application ke saath professional portfolio link share karna
Pain points:

- **Resume boring aur static hai.**
- **Projects properly showcase nahi hote.**
- **Portfolio website banana nahi aata.**
- **Har time resume update karna annoying hai.**
- **Projects ke GitHub/live links scattered hain.**

How StackFolio helps:

- **Resume information ko structured profile mein converts karta hai.**
- **Projects aur skills editable banata hai.**
- **Professional template apply karta hai.**
- **One public URL deta hai.**
- **Readiness score se missing elements batata hai.**

Persona 2: Beginner freelancer

Name: Rohan Patil
Age: 22
Profile: Student freelancer / web designer
Goal: Client ko apna work aur services professionally dikhana
Pain points:

- **Website banane ka time nahi.**
- **Work screenshots, project details and links organize nahi hain.**
- **A good-looking website design nahi kar pata.**
- **Har new project ke baad portfolio update nahi karta.**

Persona 3: Career-cell coordinator

Name: Mrs. Mehta
Profile: College placement cell coordinator
Goal: Students ke portfolios ko review/share karna
Pain points:

- **Hundreds of student resumes inconsistent format mein aate hain.**
- **Projects and technical skills quickly evaluate karna difficult hota hai.**
- **Student ke actual work links easily nahi milte.**

User journey

Tumhara demo is flow par based hona chahiye:

```text
Landing page
   ↓
Sign up / Login
   ↓
Upload resume or choose “Build manually”
   ↓
Resume data is extracted into an editable draft
   ↓
User reviews and corrects name, education, projects, skills and links
   ↓
User chooses a portfolio template
   ↓
Live preview updates
   ↓
Portfolio Readiness Score gives improvement suggestions
   ↓
User publishes portfolio
   ↓
Public portfolio opens using unique link
```

Demo fallback

Hackathon demo mein resume extraction fail ho sakta hai. Isliye dashboard par ek option hona chahiye:

> **“Try with demo profile”**

Is demo profile mein complete data ho:

- **Professional headline.**
- **About section.**
- **3 projects.**
- **8–10 skills.**
- **Education.**
- **Certifications/achievements.**
- **GitHub and LinkedIn.**
- **Project images or placeholders.**

Judge ke saamne smooth demo ke liye tum is profile ka use kar sakte ho.

MVP and feature priority

Level 1: Non-negotiable

Ye features working hone chahiye:

- **Sign up/login or demo mode.**
- **Basic profile information form.**
- **Projects, skills, education and achievements ka add/edit/delete.**
- **At least two portfolio templates.**
- **Live preview.**
- **Responsive public portfolio.**
- **Publish/unpublish status.**
- **Unique shareable portfolio URL.**

Level 2: Strong differentiators

Core app stable hone ke baad:

- **Rule-based Portfolio Readiness Score.**
- **Suggestions such as “Add GitHub link,” “Add two more skills,” or “Add outcomes to your project.”**
- **Section visibility controls.**
- **Simple project reordering.**
- **Dark/light mode in portfolio.**
- **Profile photo upload.**
- **PDF resume upload and store.**

Level 3: Brownie points

Only if everything else is already deployed and tested:

- **AI-written “About me” draft.**
- **AI improvement for a weak project description.**
- **Resume PDF text extraction.**
- **Portfolio download as PDF.**
- **Portfolio visitor count.**
- **Share via WhatsApp/LinkedIn button.**
- **Additional templates.**

What makes StackFolio different

Generic “resume-to-website” solutions se differentiate karne ke liye tumhara message ye hona chahiye:

Common resume builder	StackFolio
Produces another PDF	Produces a living public portfolio
Information remains fixed	Every section can be edited and reorganized
Focuses only on layout	Also improves readiness and completeness
Design skill is needed	Templates create a professional output
Resume is shared manually	User gets a public shareable portfolio link
No guidance after creation	Readiness score gives actionable next steps

Your unique feature: Portfolio Readiness Score

Ek simple, credible scoring system bana sakte ho:

Portfolio item	Marks
Name and professional headline	10
About section	10
Contact or social link	10
Education	10
At least one project	20
Project includes GitHub/live link	15
At least five skills	10
Achievement/certificate	10
Profile image	5

Readiness score=min(100,sum of completed criteria)

User ko only score nahi, action items bhi do:

> **“Your portfolio is 70% ready. Add a GitHub link to your projects and write measurable outcomes to improve recruiter trust.”**

This directly supports the problem’s optional portfolio-quality feedback feature.

Scope boundaries

Ye decision abhi lock karna hai:

Build karna hai

- **Editable structured profile.**
- **Projects, skills, education, achievements.**
- **Two professional templates.**
- **Live preview.**
- **Public URL.**
- **Publish/unpublish.**
- **Responsive design.**
- **Readiness score.**
- **Demo profile and robust demo flow.**

Abhi build nahi karna

- **Perfect OCR from every PDF layout.**
- **LinkedIn data import.**
- **Complex drag-and-drop editor.**
- **Payment plans.**
- **Multiple languages.**
- **Employer dashboard.**
- **Team collaboration.**
- **Full social network.**
- **Complex analytics.**
- **Email notification system.**

Ye boundaries rakhoge toh app complete lagega, half-built nahi.

Presentation narrative

Tumhari presentation mein ek real student story use karo:

> **“Meet Aarya, a final-year BCA student. She has skills, projects and certificates, but they are trapped inside a two-page resume. Recruiters cannot easily explore her GitHub work or live projects, and Aarya does not know how to build a portfolio website. With StackFolio, she transforms that resume into an editable public portfolio in minutes.”**

Demo flow:

1. **Aarya ka existing resume/profile data open karo.**
2. **Aarya ka project edit karo aur GitHub link add karo.**
3. **Portfolio template switch karo.**
4. **Readiness score 72 se 87 tak increase dikhao.**
5. **Publish button click karo.**
6. **New tab mein public portfolio URL dikhao.**
7. **Mobile responsive view dikhao.**

Haan, har user ka portfolio link unique hona chahiye, lekin portfolio edit/update hote hi new link create karna generally best practice nahi hai. Stable unique URL rakho—content update hote hi usi link par latest portfolio dikhna chahiye. Isse recruiters, GitHub, LinkedIn, WhatsApp ya resume par share kiya gaya link kabhi break nahi hoga.

Example: `https://stackfolio.vercel.app/p/aarya-shah-x7k2`
Aarya apna bio, projects, skills ya theme change karegi toh isi URL par updated portfolio automatically render hoga. Problem statement bhi editable portfolio aur unique shareable public link expect karta hai; stable link user ke liye zyada practical solution hai.

Link strategy

Recommended public URL format

Use:

```text
/p/:slug
```

Examples:

```text
/p/rahul-sharma-k8d4
/p/aarya-shah-m2p9
/p/sneha-patil-v7q1
```

Is format ke advantages:

- **Short and professional dikhta hai.**
- **Portfolio route clear hota hai.**
- **`username` collision ka risk unique suffix se solve hota hai.**
- **Current user easily rename/change slug kar sakta hai.**
- **Public route frontend mein easy hai.**
- **Vercel par deployment ke saath straightforward hota hai.**

Slug structure

```text
full-name + "-" + 4 random characters
```

For example:

```text
aarya-shah-r4x9
```

Database mein unique constraint use karo:

```text
public_slug text unique not null
```

Why update ke baad new link nahi banana chahiye

Stable link	Every update = new link
Recruiter always latest portfolio dekhega	Old links outdated/broken ho jayenge
Resume, LinkedIn, GitHub par same link rehta hai	User ko har jagah link replace karna padega
Professional and familiar behaviour	Confusing user experience
Analytics future mein easy hoti hai	Different links ka data fragment ho jayega
Simple database design	Unnecessary version-management complexity

Agar tum innovation feature add karna chaho toh version history bana sakte ho:

```text
Current published portfolio:
stackfolio.app/p/aarya-shah-r4x9

Optional snapshot:
stackfolio.app/p/aarya-shah-r4x9?version=3
```

Lekin current hackathon scope mein version history mat build karo. Public profile ka content update hone par same stable link updated data dikhaye—yehi professional approach hai.

Publish model

User ke portfolio ke states:

```text
Draft → Published → Updated → Republished / Unpublished
```

- **Draft: Public URL par visible nahi.**
- **Published: Unique link active; public can view it.**
- **Updated: User dashboard par changes karega, public link par immediately/latest content dikhega.**
- **Unpublished: Public URL disabled; visitor ko "This portfolio is not available" message milega.**

Ek optional enhancement:

```text
Save draft
Preview changes
Publish changes
```

Isse user live portfolio ko overwrite karne ke pehle preview kar sakta hai. But MVP mein edits auto-save + public update, ya simple "Save changes" sufficient hai.

Product scope lock

Ab StackFolio ki vision clear hai:

A web platform for students and freshers that transforms resume information into an editable, responsive, and shareable portfolio—using templates, guided quality feedback, and a stable public URL.

Target users

- **College students.**
- **Freshers.**
- **Internship/job seekers.**
- **Entry-level developers, designers, writers, analysts and freelancers.**

Core flow

```text
Landing Page
   ↓
Sign up / Login
   ↓
Upload resume PDF OR build profile manually
   ↓
Extract/organize information into editable fields
   ↓
Review and correct profile content
   ↓
Choose light or dark template
   ↓
Laptop/mobile preview mode
   ↓
Portfolio readiness score + suggested improvements
   ↓
Save and publish
   ↓
Receive one permanent public portfolio link
   ↓
Any future edits update content at the same link
```

MVP features

Essential build

- **Supabase authentication.**
- **Profile photo upload.**
- **Manual profile builder.**
- **Resume PDF upload and saved file.**
- **Resume text extraction if feasible.**
- **Editable sections:**
    - **Basic profile.**
    - **About.**
    - **Skills.**
    - **Education.**
    - **Projects.**
    - **Achievements/certificates.**
    - **Contact/social links.**
- **Template selector:**
    - **Dark developer style.**
    - **Light professional/corporate style.**
- **Desktop preview.**
- **Mobile device preview toggle using PC and phone SVG icons.**
- **Responsive public portfolio page.**
- **One unique permanent public URL.**
- **Publish/unpublish.**
- **Live portfolio preview.**
- **Readiness score and actionable suggestions.**
- **Vercel deployment.**

If time remains

- **AI professional summary generator.**
- **AI project-description improvement.**
- **Resume parser that maps PDF text into profile fields.**
- **Drag-and-drop section reorder.**
- **Export portfolio as PDF.**
- **Share buttons.**
- **Portfolio visit count.**
- **Third template.**
- **Custom slug editing with availability check.**

Preview design

Tumhara laptop/mobile preview idea strong hai. Editor page ke right side ya top section mein preview controls rakho:

```text
[ Desktop SVG ]  [ Mobile SVG ]

Selected: Desktop
→ Wide preview canvas, desktop layout

Selected: Mobile
→ Centered phone-frame canvas, mobile width
```

Important: Dono preview same real portfolio component se render hon. Sirf container width change karo—alag desktop/mobile UI manually maintain mat karo.

Recommended widths:

```text
Desktop preview: 100% width, max 1280px
Mobile preview: 390px width
```

Mobile mode mein:

- **Preview wrapper ke around phone-style border.**
- **`overflow-y: auto`.**
- **Page actual responsive CSS use kare.**
- **Public portfolio route ko real mobile browser width par bhi test karna.**

Resume upload and extraction

Dono options rakhna bilkul correct choice hai—lekin implementation priority clear honi chahiye:

1. **Manual builder must work fully.**
2. **Resume PDF upload file storage mein save ho.**
3. **Extracted raw text display ho, if parser works.**
4. **AI/automatic mapping to fields only after core app is stable.**

Reason: User ka real control editable dashboard se aata hai. Extraction output inaccurate ho sakta hai, isliye review-and-correct design

important hai. Problem statement user ko information edit/review/add/remove/reorder karne ka expectation deta hai.

UX copy

Upload screen par ye line use karna:

> “Upload your resume to pre-fill your profile. You can review and edit every detail before publishing.”

Isse automated extraction ki limitations professional way mein communicate hoti hain.

Documentation structure

Tumhare 10 files excellent hain. Inhe ek dedicated `/docs` folder mein rakho:

```text
/docs
  PRD.md
  Architecture.md
  database-schema.md
  api-contract.md
  design.md
  phases.md
  test-plan.md
  demo-script.md
  security-checklist.md
  AGENTS.md
```

`AGENTS.md` ko repository root mein bhi rakh sakte ho, kyunki AI tools often root-level instructions better detect karte hain:

```text
/AGENTS.md
/docs/PRD.md
/docs/Architecture.md
...
```

File ownership

File	What it should control
`PRD.md`	Problem, users, success goals, scope, features and non-goals
`Architecture.md`	Stack, project structure, routes, data flow, deployment
`database-schema.md`	Tables, columns, relationships, indexes, RLS policies
`api-contract.md`	Supabase/database actions, inputs, output and error states
`design.md`	Visual system, pages, templates, UI states, preview behaviour
`phases.md`	Feature priority and day-wise build plan
`test-plan.md`	Manual tests for user journeys, permissions, mobile and deployment
`demo-script.md`	Demo data, pitch order, click-by-click presentation flow
`security-checklist.md`	Secrets, RLS, validation, data privacy, deployment checks
`AGENTS.md`	Rules Antigravity must follow before changing the code